

CURSO ENGENHARIA DA COMPUTAÇÃO

Autorizado/Reconhecido pela Portaria nº [PREENCHER], de [PREENCHER], publicada no Diário Oficial da União em [PREENCHER].

Curso: Sistemas de Informação	Período/Turma: [PREENCHER]
Disciplina: Linguagens Formais e Compiladores	Carga Horária: 30 horas
Código da Disciplina: [PREENCHER]	
Período Letivo: 2º Semestre Ano Letivo: 2026 Classificação: Presencial Docente(s): [PREENCHER]	

OBJETIVOS DO CURSO

O objetivo geral do curso é a formação de profissionais para o desenvolvimento científico e tecnológico da computação, com habilitação para especificar, projetar, implementar, manter e avaliar sistemas de computação voltados à solução de problemas de natureza diversificada. Busca-se formar profissionais qualificados e versáteis em tecnologia da informação, capazes de contribuir para o sucesso das organizações por meio do uso estratégico da informática e da gestão de sistemas de informação.

EMENTA

Gramáticas formais e autômatos. Gramáticas regulares e autômatos finitos. Gramáticas livres de contexto. Gramáticas sensíveis ao contexto. Linguagens recursivas. Organização e estrutura de compiladores. Análise léxica. Análise sintática. Alocação e gerência de memória. Formas internas de programas-fonte. Análise semântica. Geração de código. Otimização de código.

COMPETÊNCIAS

1. Selecionar, configurar e gerenciar tecnologias da informação nas organizações;
2. Atuar em organizações públicas e privadas, utilizando tecnologias da informação para o alcance dos objetivos organizacionais;
3. Identificar oportunidades de melhoria e projetar soluções computacionais adequadas às necessidades organizacionais;
4. Comparar soluções alternativas, considerando desempenho, integração, riscos e adequação técnica;

5. Modelar e implementar soluções de tecnologia da informação em diferentes domínios de aplicação;
6. Aplicar conceitos, métodos, técnicas e ferramentas de desenvolvimento de software e gerenciamento de projetos;
7. Representar formalmente problemas e soluções computacionais por meio de modelos, linguagens e abstrações;
8. Analisar linguagens de programação e seus mecanismos de tradução, execução e otimização;
9. Desenvolver componentes de compiladores e interpretadores com correção, clareza e rastreabilidade;
10. Gerenciar o desempenho, a qualidade e a evolução de aplicações e sistemas de informação.

OBJETIVOS DE APRENDIZAGEM

1. Classificar linguagens formais segundo a hierarquia de Chomsky, relacionando cada classe aos seus modelos de reconhecimento.
2. Construir gramáticas regulares e autômatos finitos para representar e reconhecer linguagens especificadas em problemas propostos.
3. Converter, quando aplicável, expressões regulares, gramáticas regulares e autômatos finitos entre representações equivalentes.
4. Elaborar gramáticas livres de contexto e analisar derivações, árvores sintáticas, ambiguidade e propriedades de linguagens.
5. Distinguir gramáticas sensíveis ao contexto e linguagens recursivas, identificando seus limites de expressividade e decidibilidade.
6. Descrever a organização de um compilador e relacionar suas fases, entradas, saídas e estruturas de dados internas.
7. Projetar e implementar um analisador léxico capaz de reconhecer tokens e tratar erros léxicos em uma linguagem simplificada.
8. Aplicar técnicas de análise sintática para construir ou interpretar árvores sintáticas e diagnosticar erros de sintaxe.
9. Executar verificações semânticas, incluindo tipos, escopos, declarações e uso de identificadores, com apoio de tabela de símbolos.
10. Representar programas-fonte em formas intermediárias adequadas à análise, à geração e à otimização de código.
11. Explicar e aplicar estratégias básicas de alocação e gerência de memória em tempo de compilação e execução.
12. Gerar código-alvo ou intermediário para construções elementares e aplicar otimizações locais preservando a semântica do programa.
13. Integrar as etapas de um compilador didático, documentando decisões de projeto, testes, limitações e resultados.

CONTRIBUIÇÃO PARA O PERFIL DO EGRESSO

A disciplina contribui para o desenvolvimento do raciocínio lógico e abstrato, da capacidade de modelagem e da compreensão dos fundamentos matemáticos e computacionais que sustentam linguagens de programação, compiladores e ferramentas de desenvolvimento. Ao construir autômatos, gramáticas e componentes de um compilador, o estudante exercita análise de problemas, especificação formal, implementação, testes, documentação técnica e tomada de decisão baseada em critérios de correção, eficiência e manutenibilidade. Esses conhecimentos favorecem a atuação em engenharia de software, desenvolvimento de linguagens, processamento de texto, segurança, inteligência artificial e construção de ferramentas computacionais.

CONTEÚDO

1. Conceitos de alfabeto, palavra, linguagem, operações sobre linguagens e problemas de decisão.
2. Hierarquia de Chomsky e classes de gramáticas formais.
3. Gramáticas regulares, expressões regulares e linguagens regulares.
4. Autômatos finitos determinísticos e não determinísticos; transições vazias.
5. Conversões entre expressões regulares, gramáticas regulares e autômatos; minimização de autômatos.
6. Gramáticas livres de contexto: derivações, árvores sintáticas, ambiguidade e formas normais.
7. Autômatos com pilha e reconhecimento de linguagens livres de contexto.
8. Gramáticas sensíveis ao contexto, linguagens recursivas e noções de decidibilidade.
9. Organização geral de compiladores: front-end, middle-end e back-end.
10. Análise léxica: tokens, padrões, autômatos, tratamento de erros e implementação.
11. Análise sintática: descendente, ascendente, árvores sintáticas e recuperação de erros.
12. Formas internas de programas-fonte: árvore sintática abstrata e código intermediário.
13. Análise semântica: tipos, escopos, declarações, tabela de símbolos e verificações contextuais.
14. Alocação e gerência de memória: estática, pilha, heap e registros de ativação.
15. Geração de código para expressões, comandos, controle de fluxo e chamadas de procedimentos.
16. Otimização de código: simplificação, propagação de constantes, eliminação de código morto e otimizações locais.
17. Integração das fases e desenvolvimento de um compilador didático.

METODOLOGIA DE ENSINO

A disciplina será desenvolvida por meio de aulas expositivas dialogadas, resolução de exercícios, estudos dirigidos, aprendizagem baseada em problemas, atividades em equipe, análise de exemplos de linguagens, construção de gramáticas e autômatos, implementação de algoritmos e desenvolvimento incremental de um compilador didático. Serão utilizados quadros de derivação, diagramas de estados, árvores sintáticas, tabelas de símbolos, representações intermediárias e ferramentas computacionais para testar e validar modelos.

As atividades práticas incluirão construção e simulação de autômatos, especificação de analisadores, implementação de módulos léxicos, sintáticos e semânticos, geração de representações intermediárias e aplicação de otimizações simples. Os estudantes produzirão registros técnicos, códigos-fonte, relatórios e apresentações, relacionando teoria formal, implementação e aplicações profissionais.

Nesta disciplina, a Inteligência Artificial é considerada ferramenta de apoio à aprendizagem. O uso é RESTRITO e somente poderá ocorrer quando autorizado pelo docente, para finalidades como organização de ideias, revisão textual, geração de casos de teste e explicação preliminar de mensagens de erro. Quando autorizada, a ferramenta utilizada e sua contribuição deverão ser declaradas. A especificação formal, a autoria do código, a validação dos resultados e as conclusões permanecem sob responsabilidade integral do estudante. A geração integral de trabalhos ou códigos sem declaração constitui violação da integridade acadêmica.

CRITÉRIO DE AVALIAÇÃO

O desempenho acadêmico será acompanhado por processo avaliativo contínuo e cumulativo, alinhado aos objetivos de aprendizagem e aos conteúdos da disciplina. Todas as atividades deverão observar os princípios de autoria, originalidade e integridade acadêmica.

Disciplina com Avaliação Processual e Pontual. A Avaliação Processual corresponderá a 3,0 pontos em cada bimestre e será composta por, no máximo, dois instrumentos. A Avaliação Pontual corresponderá a 7,0 pontos em cada bimestre e será realizada individualmente, conforme o Calendário Acadêmico.

Instrumentos Avaliativos	Bimestre	Pontuação
Lista prática de gramáticas, expressões regulares e autômatos; e/ou Projeto de analisador léxico e relatório técnico	1º	3,0
Estudo de caso de análise sintática e semântica e/ou Projeto integrador: compilador didático e apresentação	2º	3,0

A Avaliação Pontual será composta por questões objetivas e discursivas, totalizando 7,0 pontos em cada bimestre. A média bimestral resultará da soma da Avaliação Processual (3,0) com a Avaliação Pontual (7,0). O estudante que obtiver média inferior a 7,0 deverá prestar exame final e será aprovado se alcançar média final igual ou superior a 5,0.

BIBLIOGRAFIA BÁSICA

FURTADO, O. J. V. Apostila de Linguagens Formais e Compiladores: versão 2. Florianópolis: UFSC, 2002. Disponível em: www.inf.ufsc.br/~olinto.

MENEZES, Paulo Blauth. Linguagens formais e autômatos. 6. ed. Porto Alegre: Bookman, 2011. 256 p.

SANTOS, Pedro Reis; LANGLOIS, Thibault. Compiladores: da teoria à prática. Rio de Janeiro: LTC, 2018. Livro digital. ISBN 9788521635161. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788521635161>.

BIBLIOGRAFIA COMPLEMENTAR

COOPER, Keith D.; TORCZON, Linda. Construindo compiladores. 2. ed. Rio de Janeiro: Campus, 2014. 656 p.

LOUDEN, Kenneth C. Compiladores: princípios e práticas. São Paulo: Cengage Learning, 2004. Livro digital. ISBN 9788522128532. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788522128532>.

BARBOSA, Cynthia da Silva et al. Compiladores. Porto Alegre: SAGAH, 2021. Livro digital. ISBN 9786556902906. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9786556902906>.

SEBESTA, Robert W. Conceitos de linguagens de programação. 11. ed. Porto Alegre: Bookman, 2018. Livro digital. ISBN 9788582604694. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788582604694>.

SOUSA, Carlos Estevão Bastos et al. Linguagens formais e autômatos. Porto Alegre: SAGAH, 2021. Livro digital. ISBN 9786556901138. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9786556901138>.

LEITURAS COMPLEMENTARES

Capítulos, artigos, manuais e materiais técnicos relacionados a autômatos, gramáticas, análise léxica, análise sintática, geração e otimização de código poderão ser indicados pelo docente no Ambiente Virtual de Aprendizagem, respeitando os objetivos e a sequência didática da disciplina.

RELAÇÕES INTERDISCIPLINARES

Relação horizontal: [PREENCHER PELA COORDENAÇÃO DE CURSO, conforme as disciplinas ofertadas no mesmo período].

Relação vertical: Algoritmos e Programação; Estruturas de Dados; Matemática Discreta; Teoria da Computação; Linguagens de Programação; Programação Orientada a Objetos; Engenharia de Software; Arquitetura de Computadores; Sistemas Operacionais; Inteligência Artificial.

A articulação ocorre na formalização de linguagens, na análise de algoritmos, na implementação de estruturas de dados, na organização de memória, na tradução de programas, na validação de tipos e escopos e na construção de ferramentas de desenvolvimento de software.

CRONOGRAMA DA DISCIPLINA

Carga horária total: 30 horas, distribuídas em 20 encontros de 1h30. As datas deverão ser preenchidas conforme o Calendário Acadêmico e o dia de oferta da turma. Em caso de dia não letivo, a carga horária deverá ser reposta por atividade extraclasse orientada, com registro do conteúdo e da atividade proposta.

Sem.	Objetivos de aprendizagem, conteúdos e atividades	CH	Data
1	Apresentar o plano de ensino; definir alfabetos, palavras, linguagens e operações sobre linguagens. Classificar gramáticas e linguagens segundo a hierarquia de Chomsky.	1h30	__/__/2026
2	Construir gramáticas regulares e expressões regulares para linguagens propostas.	1h30	__/__/2026
	Construir e simular autômatos finitos determinísticos e não determinísticos.	1h30	__/__/2026
5	Converter expressões regulares, gramáticas regulares e autômatos; aplicar minimização.	1h30	__/__/2026
6	Elaborar gramáticas livres de contexto, derivações e árvores sintáticas.	1h30	__/__/2026
7	Analisar ambiguidade e aplicar formas normais em gramáticas livres de contexto.	1h30	__/__/2026
8	Relacionar gramáticas livres de contexto e autômatos com pilha. Atividade processual: lista prática.	1h30	__/__/2026
9	Distinguir gramáticas sensíveis ao contexto, linguagens recursivas e noções de decidibilidade.	1h30	__/__/2026
11	Identificar a arquitetura e as fases de um compilador; definir o projeto integrador.	1h30	__/__/2026
12	Especificar tokens, padrões e regras para análise léxica.	1h30	__/__/2026

13	Implementar e testar um analisador léxico. Atividade processual: projeto e relatório.	1h30	__/__/2026
14	Aplicar análise sintática descendente e construir árvores sintáticas.	1h30	__/__/2026
15	Analisar técnicas sintáticas ascendentes e estratégias de recuperação de erros.	1h30	__/__/2026
16	Construir formas internas de programas-fonte e tabelas de símbolos.	1h30	__/__/2026
17	Executar análise semântica de tipos, escopos e declarações.	1h30	__/__/2026
18	Analisar alocação e gerência de memória; gerar código intermediário e aplicar otimizações locais.	1h30	__/__/2026
19	Avaliação Pontual do 2º bimestre; entrega e apresentação do compilador didático.	1h30	__/__/2026
20	Realizar integração, devolutiva, revisão dos projetos e síntese da disciplina.	1h30	__/__/2026

** O cronograma poderá sofrer alterações em razão de fatores institucionais ou acadêmicos, preservando-se a carga horária e os objetivos de aprendizagem.*

Aviso Importante: Guarde este programa. Ele é um documento importante.

[NOME DA SECRETÁRIA ACADÊMICA]
Secretária Acadêmica